# CAPÍTULO 2.
# EL PARADIGMA FUNCIONAL

## 2.1 Introducción

El paradigma funcional es uno de los paradigmas más antiguos y fundamentales de la programación informática. Los orígenes teóricos del modelo funcional se remontan a la década de los 30, más precisamente al año 1934, cuando Alonso Church introdujo un modelo matemático de computación llamado cálculo lambda (o λ-cálculo), una teoría matemática elaborada por él mismo como apoyo a sus estudios sobre computabilidad.

El objetivo del paradigma funcional es conseguir una programación expresiva y matemáticamente elegante, en la que no sea necesario describir paso a paso la secuencia de computación para implementar informáticamente un problema.

En esta unidad se presentan, además del concepto de *función* con su especificación procedimental y declarativa, los siguientes conceptos que definen las características del paradigma:

- *Transparencia referencial* de una función, en el sentido de que se comporta como una función matemática; es decir, que si recibe un valor de entrada siempre producirá el mismo valor de salida.
- *Composición funcional*, que consiste en que un programa es una función compuesta de otras más simples, formando expresiones en que el resultado de una función es argumento de entrada de otra función.
- *Función de orden superior*, denominada así porque sus argumentos y valor de retorno pueden ser otra función.
- *Concepto de estado*, en el sentido de que en el paradigma funcional este concepto no existe como concepto de celda de memoria donde se almacenan los datos entrada, se transforman y se almacenan los resultados de salida.
- *Función polimórfica*. Es una función que puede asumir diferentes formas o manipular tipos diferentes de objetos. A tal efecto se describen los tres tipos de polimorfismo: sobrecarga, polimorfismo dinámico y polimorfismo paramétrico.
- *Evaluación retardada*. Este es un tipo de evaluación de una función donde sus argumentos no se evalúan hasta que no se necesitan ("*call by need*"). Es decir, los argumentos son evaluados cuando los necesita el cuerpo de la función.

También se describen: la *mónada* como elemento encapsulado que proporciona características procedimentales sin perder las propiedades del paradigma funcional, los *tipos de datos* básicos que utiliza el paradigma funcional y el concepto de *programa funcional* como un conjunto compuesto de funciones que juntas definen un objetivo específico de aplicación.

## 2.2 Objetivos

Los objetivos de la unidad son básicamente los siguientes:

- Conocer el concepto de función como elemento básico del paradigma funcional y la diferencia en especificar una función de forma declarativa o de forma procedimental.
- Conocer qué características principales definen al paradigma funcional: su transparencia referencial, su composición funcional, la utilización de funciones de orden superior, la no existencia del concepto de estado, la utilización de funciones polimórficas y la evaluación retardada o diferida.
- Conocer las ventajas e inconvenientes que presenta el paradigma funcional en relación con el imperativo.

## 2.3 El Paradigma Funcional. Concepto y Características

El paradigma funcional está basado en el modelo matemático de composición funcional. En este modelo, el resultado de un cálculo es la entrada del siguiente, y así sucesivamente hasta que la composición produce el resultado deseado. Por ejemplo, para ordenar una lista, se puede diseñar la solución como una concatenación de listas más pequeñas, cada una de las cuales ya está clasificada. Esto reduce el problema a seleccionar las listas más pequeñas.

En este paradigma no existe el concepto de celda de memoria para almacenar un dato que es asignado o modificado. Más bien, existen valores intermedios que son el resultado de cálculos anteriores y las entradas a subsiguientes cálculos. Tampoco existen sentencias imperativas y todas las funciones tienen transparencia referencial, en el sentido de que para un mismo dato de entrada a una función ésta proporciona siempre el mismo resultado.

El paradigma funcional incorpora el concepto de función como objeto de primera clase, lo que significa que las funciones pueden ser tratadas como datos (pueden pasar como parámetros, ser calculadas y devueltas como valores normales y mezcladas en el cálculo con otras formas de datos).

### 2.3.1 La Función

Un programa funcional es pues una serie de funciones que juntas definen un objeto específico de aplicación.

La función es una regla de correspondencia (o asociación) de miembros de un conjunto (el dominio) con miembros de otro conjunto (el rango). Por ejemplo, la función "cubo":

$$cubo(x) = x^3 \qquad x \in [n^o.\ naturales]$$

hace corresponder miembros del conjunto de los números naturales: 1,2,3,..., que configuran el dominio con miembros del conjunto de los números naturales: 1, 8, 27, ...., que forman el rango.

Una definición de función especifica el dominio, el rango y la regla de correspondencia de la función. Una vez definida la función, puede ser aplicada a un elemento particular del dominio y esta aplicación produce el elemento asociado dentro del rango.

El parámetro x en la función *cubo(x)* es una variable matemática que representa cualquier elemento del dominio, lo cual no es lo mismo que una variable de programación que toma diferentes valores durante la ejecución del programa.

El concepto de función, desde el punto de vista informático, es una pieza individual de codificación que siempre efectúa el mimo conjunto de operaciones, pero que puede operar con diferentes datos, y proporcionar algún tipo de resultado, que se conoce como "valor de la función".

Una función informática se define mediante una rutina que devuelve un valor: la regla de correspondencia de un valor del dominio con un valor del rango se establece en términos de un número de pasos que tienen que ser ejecutados en un orden específico mediante estructuras de control. Por otro lado las funciones matemáticas se definen funcionalmente; es decir, las reglas de correspondencia se definen en términos de aplicación de otras funciones o de ella misma (concepto de recursividad).

***Ejemplo 2.1: Sumar los elementos de una lista L***

a) En un lenguaje puramente funcional tipo Function Programming (FP) creado por John Backus se puede definir como:

```
sumar: L= if null:L then 0 else if null tail:L then first:L
      else +: <first:L, sumar: (tail:L)>
```

"Si la lista L está vacía entonces *sumar* es 0 en caso contrario si la cola de la lista L es nula, devolver la cabeza de L; si no sumar la cabeza de la lista con la llamada recursiva a *sumar* tomando como parámetro la cola de L"
Dada la lista L=<4, 2, 5>, el resultado de *sumar: L* sería 11.

b) En un lenguaje imperativo tipo C/C++ la función sumar se definiría como:

```
struct lista { int valor; lista* siguiente;};

int sumar(lista *L)
{
   lista *aux =L;
   int suma=0;
   while (aux != NULL) {
      suma += aux->valor;
      aux = aux->siguiente;
   }
   return suma;
}
```

Se aprecia en b) el carácter de rutina de la función "sumar" con sentencias de asignación de tipo destructivo.

### 2.3.2 Especificación Procedimental de la Función

La forma en que se especifica la función puede variar. En particular, se pueden especificar *procedimentalmente,* indicando una secuencia de control, o *matemáticamente* mediante su definición sin secuencia de control. En la solución procedimental u operacional se controla explícitamente el orden en que se efectúa el cálculo. Así, al ordenar una lista, primero debemos determinar si la lista está vacía y realizar el tratamiento en el caso de que no lo esté. Este orden de cálculo origina una *sobrespecificación* que es lo que caracteriza a las soluciones procedimentales. En este estilo procedimental, se define el problema como una construcción en la que se describe una secuencia de etapas que aplicará composición funcional para obtener el resultado deseado.

En la práctica, los lenguajes que soportan esta forma procedimental del paradigma funcional a menudo incluyen construcciones imperativas, tal como la asignación, que destruyen el natural no-efecto de lado de este paradigma y obligan a considerar la secuencia del programa en su construcción.

Un ejemplo es COMMONLISP, considerado como el estándar del LISP. COMMONLISP se ha desarrollado como un lenguaje funcional procedimental que admite ambos tipos de construcciones: la composición funcional con transparencia referencial, y la asignación de variables con secuencias de control en la ejecución del programa.

### 2.3.3 Especificación Declarativa de la Función

En la solución declarativa, el paradigma se ajusta al modelo matemático de función, definiendo las funciones de la misma forma a como se realiza en el cálculo matemático (Ambler, 92):

$f(v) = \quad cond_1(x),\ expr_1(x)$

$\quad\quad cond_2(x),\ expr_2(x)$

$\quad\quad \ldots\ldots\ldots\ldots\ldots\ldots$

$\quad\quad cond_n(x),\ expr_n(x)$

Para un valor particular de entrada x se cumplirá una de las condiciones del dominio de entrada y se devolverá el resultado de la expresión correspondiente. Dado este comportamiento, no es necesario especificar el orden de evaluación de las condiciones. Por ejemplo, la función *par(r)* que se describe a continuación es una solución funcional declarativa porque se especifican dos transformaciones disjuntas sin orden de ejecución:

$$\text{par}(x) = \begin{cases} \text{resto}(x,2) == 0 \rightarrow \text{true} \\ \text{resto}(x,2) != 0 \rightarrow \text{false} \end{cases}$$

$$\text{resto}(x,y) = x - (x//y) * y$$

Mientras que la función *par(x)* que se describe a continuación es una solución funcional procedimental porque se indica el orden de ejecución de las sentencias:

*par(x) = if ((x - (x//2) * 2) = = 0) then true* **else** *false*

Es decir, en el estilo procedimental se define el problema como una construcción, en la que se describe una secuencia de etapas que aplicarán composición funcional para obtener el resultado deseado, mientras que en el estilo declarativo se define el problema como una colección de *transformaciones disjuntas* que, tomadas colectivamente, definen computacionalmente una función.

Por otra parte, si se flexibilizan los requerimientos que mantienen las condiciones de las funciones *condi(x)*, y el orden de evaluación es de nuevo importante, se pasa del paradigma funcional declarativo al "pseudodeclarativo" y, en este caso, no se puede afirmar si un conjunto particular de condiciones es disjunto; por lo que la mayor parte de los lenguajes funcionales definen el orden de evaluación y dejan al programador que compruebe que las condiciones son disjuntas o que están ordenadas apropiadamente.

La distinción procedimental o declarativa de la función estriba fundamentalmente en si está especificada explícitamente la *secuencia* de ejecución. Con el estilo procedimental el desarrollador del software es responsable de definir la secuencia de instrucciones, incluido el fin de la ejecución. Por contraste, el estilo declarativo prevé la terminación sin requerir un control explícito de la ejecución. Esta diferencia es significativa para diferenciar ambos modelos de programación.

### 2.3.4 Características del Paradigma Funcional

Las características principales que definen este paradigma y que describiremos a continuación son:

- Aplica funciones con *Transparencia Referencial*
- Aplica la *Composición Funcional*
- Utiliza funciones de *Orden Superior*
- No existe el concepto de *Estado*
- Utiliza funciones con *Polimorfismo Paramétrico*
- Utiliza evaluación *Retardada o Diferida*

## 2.4 Transparencia Referencial

La diferencia más importante entre una función matemática y una función informática proviene de la noción de *variable modificable*. Los parámetros de una función matemática simplemente representan valores que se fijan en el tiempo de aplicación, donde su aplicación origina otro valor. Mientras que los parámetros de una función informática son nombres de celdas de memoria que pueden ser cambiados o no en la ejecución de la función, produciendo un efecto lateral. Como consecuencia de este efecto lateral, las funciones informáticas, a diferencia de las matemáticas, no siempre pueden ser combinadas como *expresiones*.

Cuando no existe efecto lateral se dice que la función tiene *transparencia referencial*. Ésta es una propiedad fundamental de las funciones matemáticas, ya que permite considerarlas como cajas negras y utilizarlas en composición funcional. Se cumple que cada expresión proporciona un *único* valor que no puede cambiar por la forma de evaluar la expresión o por permitir que diferentes partes de un programa compartan la expresión. La evaluación de la expresión simplemente cambia la forma de la expresión pero nunca su valor.

Es decir, una función tiene transparencia referencial si se comporta como una función matemática, en el sentido de que si recibe un valor de entrada siempre producirá el mismo valor de salida, y que el resultado del cálculo de la función únicamente se devuelve por el nombre de la función y no por alguno de sus argumentos.

Se va a ver esto con algunos ejemplos que aclaren este concepto.

***Ejemplo 2.2: El mismo valor de entrada no produce un mismo valor de la función***

Supóngase que se tiene la función *fun(x)* con un conmutador o switch *sw* que cambia de valor de cierto a falso cada vez que se ejecuta la función y que afecta al resultado que devuelve la función, de acuerdo con los siguientes términos:

fun (x): x, si sw= cierto

$x^2$, si sw= falso

y que está codificada en C con el siguiente código:

```
#include<iostream>
using namespace std;
int sw;
int fun (int n);

void main()
{
   sw=0;
   cout << fun(2)+fun(3) << endl;
   cout << fun(3)+fun(2) << endl;
}

int fun (int n)
{
    int v;
    if (sw) v=n; else v=n*n;
    sw= !sw;
    return v;
}
```

Evidentemente esta función no tiene transparencia referencial, porque la primera vez muestra el valor 11 (2+3*3) y la segunda el valor 7 (3+2*2), lo que sería muy sorprendente para cualquier matemático que no tuviera idea de informática, pues la suma tiene la propiedad conmutativa por lo que para él sería totalmente sustituible fun(2)+ fun(3) por fun (3)+ fun (2).

La base del problema radica en la instrucción de asignación "sw = !sw" que en cada ejecución de la función cambia el valor de sw.

Otro ejemplo de falta de transparencia referencial sería la función "Cuadrado" que se define a continuación:

***Ejemplo 2.3: Retorna el valor de la función por un argumento***

*Cuadrado(x,y) = {y=x*x, $\forall x \in \Re$}*

*Cuadrado* no tiene transparencia referencial porque el "valor de retorno" de la función se devuelve mediante la variable *y*, y no mediante el nombre de la función. Realmente, debería ser *Cuadrado (x)= x * x*. Su codificación en C++ es:

```
#include <iostream>

void cuadrado (int x, int &y)
{
   y = x*x;
}
```

```
int main()
{
   using namespace std;
   int a;
   cuadrado(3,a);
   cout << a << endl;
   cuadrado(5,a);
   cout << a << endl;
}

Resultado: 9 y 25
```

Para finalizar, se presenta otra variante de falta de transparencia referencial de una función. Es el caso en que existe dentro de la función una variable fija, es decir, estática, que se incrementa en cada ejecución de la función.

***Ejemplo 2.4: Retorna el valor de la función usando una variable global***

*fun (x)= { 2(x+y), $\forall x,y \in \Re$, siendo y una variable estática}.*

Cuya codificación en C++ sería:

```
#include <iostream>

int fun (int x)
{
   static int y=0;
   y=y+1;
   return 2* (x+y);
}

int main()
{
   using namespace std;
   cout << fun(3) << endl;
   cout << fun(3) << endl;
   cout << fun(3) << endl;
}

Resultado: 8, 10, 12
```

La primera vez que se llama a *fun*, la variable *y* se inicializa a 0, pero esta acción no se vuelve a realizar en las próximas llamadas donde *y* conserva su último valor.

Desde el punto de vista matemático resulta incomprensible que la misma llamada a la función *fun* con el mismo argumento x=3, dé resultados diferentes. Ello se debe a que no existe transparencia referencial y *fun* realiza acciones diferentes en cada llamada.

Se ve pues, que la transparencia referencial es muy importante para no distorsionar el carácter de una función, de modo que se sepa que al llamarla con los mismos valores de los parámetros

siempre va a obtener el mismo valor de resultado, y este valor, única y exclusivamente, va a ser devuelto directamente por la función.

La transparencia referencial es indispensable para la *composición* de funciones ya que reemplaza "valores por valores", sustituyendo iguales por iguales, permitiendo con ello su transformación automática en formas equivalentes que son más eficientes para el computador. También permite *verificar la corrección* de los programas facilitando la construcción y prueba de asertos para comprobar su corrección.

## 2.5 Composición Funcional

Como se ha señalado al principio, el paradigma funcional se basa en el modelo matemático de composición funcional, que consiste en formar expresiones en que el resultado de una función es argumento de entrada de otra función y así sucesivamente. Un programa es una función (o grupo de funciones), normalmente compuestas de otras más simples. La relación entre funciones es muy simple: una función puede llamar a otra, o el resultado de una función se puede usar como un argumento de otra función [Watt, 1990].

Para que exista composición funcional es necesario que las funciones tengan transparencia referencial, hecho que ocurre en el paradigma funcional.

Supongamos, como ejemplo, que se quiere calcular "la suma de los números comprendidos entre 0 y el cuadrado del número mayor leído *x* o *y*, utilizando tres funciones: una que halle el mayor, otra el cuadrado y otra que obtenga la suma". Matemáticamente, la función "sumarnumeros" sería de la forma:

$$sumarnumeros(x,y) = \begin{cases} x \geq y \rightarrow \sum_{0}^{x} i \\ x < y \rightarrow \sum_{0}^{y} i \end{cases}$$

Y codificada funcionalmente en Hope sería el ejemplo 2.5, en que se han definido las funciones: *mayor(x,y)* que proporciona el valor mayor entre x e y; *cuadado(x)* que obtiene $x^2$; *suma(x)* que de modo recursivo obtiene la suma 1+2+3+...+x; y *sumarnumeros(x,y)* como composición funcional de las tres funciones anteriores.

***Ejemplo 2.5: Función sumar números en Hope***

```
dec  mayor:  num # num -> num ;
--- mayor ( x, y) <= if ( x >= y) then  x  else  y;

dec  cuadrado:  num-> num ;
--- cuadrado( x ) <= x*x ;

dec  suma: num-> num;
---  suma (0) <= 0;
---  suma (x) <= suma (x-1) +x;

dec sumarnumeros: num # num  -> num ;
--- sumarnumeros(x,y) <= suma (cuadrado(mayor(x,y)));
```

```
Ejecución: >> sumarnumeros(12,5);
Resultado: >>: 10440
```

En la programación funcional la ejecución es interpretativa, de forma que cada vez que el usuario introduce una acción, ejemplo *sumarnumero(12,5)*, el sistema la ejecuta, traduciendo las funciones que implican esa acción, hasta producir el resultado correspondiente, en este caso 10440.

Los lenguajes imperativos también aceptan composición funcional, pudiendo escribirse expresiones del tipo *suma (cuadrado(mayor(x,y)))*, pero para que ello proporcione resultados adecuados es necesario que estas funciones tengan transparencia referencial.

En resumen, la *composición funcional* es la forma de programar funcionalmente: Se tiene una serie de funciones y se compone un programa construyendo *otra función* más compleja que utiliza las anteriores, de forma que el resultado de una sea la entrada de otra. La ejecución del programa es la ejecución de esta función compuesta.

Debido a esta forma de programar, y de ejecutar el programa, ello plantea (aunque no es necesario) una ejecución *interactiva* entre el usuario y el programa. El usuario pide la ejecución de la función compuesta dando un valor a los argumentos de entrada y el programa le responde con el resultado de esa ejecución.

## 2.6 Funciones de Orden Superior

Hemos comentado que la programación funcional incorpora el concepto de función como objeto de primera clase, indicando con ello que las funciones pueden ser tratadas como datos (pueden pasarse como parámetros, ser calculadas y devueltas como valores normales, y mezcladas en el cálculo con otras formas de datos).

Una función cuyos parámetros y valores de retorno no son funciones se denomina *función de primer orden*, mientras que la que admite dichas características se denomina *función de orden superior o de primera clase* o también *función funcional.*

A continuación se presentan dos ejemplos que aclaran este concepto.

Supongamos que se quiere diseñar la función "Calcularrec" que permite aplicar un cálculo determinado (por ejemplo el área o el perímetro) a un rectángulo a partir de los valores de sus lados.

En un lenguaje funcional tipo *Hope* se declararían y definirían las funciones de primer orden *area(x,y)* y *perimetro(x,y)* que recibirían como argumentos dos números enteros y devolverían el valor del área o del perímetro respectivamente; y la función de orden superior *calcularrec(x,y,f)* que recibiría el valor de los lados *x* e *y* y una función *f* que sería el *área* o el *perímetro* a calcular, devolviendo el resultado de la función incorporada como argumento. El ejemplo 2.6 proporciona la codificación en Hope de este ejemplo.

***Ejemplo 2.6: Cálculo del área o el perímetro de un rectángulo mediante una función de orden superior, codificada en Hope***

```
dec area: num # num -> num;
--- area(x,y) <= x*y;
```

```
dec perimetro: num # num -> num;
--- perimetro(x,y) <= 2*(x+y);
dec calcularrec: num # num # (num # num -> num) -> num;
--- calcularrec(x,y,f) <= f(x,y);

>> calcularrec(3,4,area);
>>: 12
>> calcularrec(3,4,perimetro);
>>: 14
```

En programación procedimiental u operacional no se definen funciones de orden superior en sentido estricto pero se pueden utilizar aplicando punteros. Y el ejemplo anterior quedaría codificado en C++ como se indica en el ejemplo 2.7.

***Ejemplo 2.7: Cálculo del área o el perímetro de un rectángulo mediante una función de orden superior, codificada en C++***

```
#include<iostream>
// Tipo que representa funciones de dos argumentos que devuelven
entero
typedef long (* TFuncion) (int, int);
long area (int x, int y) { return x*y; }
long perimetro( int x, int y) { return 2*(x+y);}
long calcularrec ( int a, int b, TFuncion pf) { return pf(a ,b); }

// Programa principal
int main() {
   using namespace std;
   int i, a, b; cin>>i>>a>>b;
 // si i!=0 calcula el  área, si no el perímetro.
   if ( i )
      cout<< calcularrec (a, b, area) <<endl;
   else
      cout<<calcularrec (a, b, perimetro)<<endl;
}
Entrada: 1 3 4 / 0 3 4                    Resultado: 12 / 14
```

En este caso, la función *calcularrec* se comporta como una función de orden superior con la diferencia básica de que su tercer argumento *pf* no es una función "real" sino un *puntero* a una función.

El carácter de orden superior es una característica fundamental que tienen que tener las funciones que participen en la construcción de un *programa funcional*.

Normalmente se utilizan funciones de orden superior para hacer un código reutilizable y para representar datos de un modo primario (Watt, 1990).

Permitir que una función retorne otra función como su resultado abre interesantes posibilidades que son utilizadas con intensidad en la programación funcional. Así, el modo más simple de crear un nueva función es componiendo dos funciones ya existentes utilizando esta propiedad de funciones de orden superior.

Esta capacidad se puede utilizar para expresar *modelos comunes* de recursión y permitir que las funciones recursivas sean expresadas utilizando aplicaciones no recursivas de funciones de orden superior. Esta técnica conduce a programas concisos y abstractos que son sorprendentemente cortos teniendo en cuenta la compleja computación que realizan.

## 2.7 Concepto de Estado

En el paradigma funcional no existe el concepto de *estado* como concepto de celda de memoria, donde se almacenan los datos de entrada, se realizan transformaciones sobre ellos y se almacenan los resultados de salida. Lo que existen son valores locales que almacenan los resultados de cálculos intermedios.

Como consecuencia de ello:

- No se pueden realizar *asignaciones destructivas* del tipo a ← b.
- No se pueden realizar *procesos iterativos* del tipo: do, while, for,..etc.

A continuación se presenta el ejemplo de la función *factorial* codificada procedimentalmente con estructura iterativa y funcionalmente con estructura recursiva.

***Ejemplo 2.8: Función factorial procedimental en C++***

```
/* Función factorial en C++ */
#include<iostream>
long factorial (int x) {
      long total =1;
      while(x>0){
            total *= x;
            x--;
            }
      return total;
}
int main() {
      using namespace std;
      int dat;
      cin >> dat;
      cout << factorial(dat) << endl;
}
```

Vemos que la función factorial utiliza la variable *total* para, mediante asignación destructiva, ir almacenando el valor del *factorial(x)*. Por otra parte, utiliza la estructura iterativa *while* para disminuir el valor de x.

***Ejemplo 2.9: Función factorial declarativa en Hope***

```
!Función factorial en Hope
dec factorial: num -> num;
--- factorial (x) <= if x=<0 then 1 else x * factorial (x-1);
```

Vemos, en este caso, la forma más simple y elegante de definir la función factorial utilizando una estructura recursiva.

## 2.8 Funciones Polimórficas

De acuerdo con el diccionario de la Real Academia Española, la palabra "polimorfismo" representa "la calidad o estado de un elemento de asumir diferentes formas"; es decir, es la capacidad de una entidad de tomar varias formas.

En programación el polimorfismo indica que el mismo componente del lenguaje puede asumir diferentes tipos o manipular tipos diferentes de objetos. Como Cardelli y Wegner señalan [Cardelli, 1985], "los lenguajes convencionales, tales como Pascal, están basados en la idea de que las funciones, los procedimientos y los operandos tienen un único tipo. Tales lenguajes se dice que son *monomórficos*, en el sentido de que cada valor y variable es interpretada para pertenecer a un único tipo. Los lenguajes de programación monomórficos se pueden contrastar con los lenguajes polimórficos en que algunos valores y variables pueden tener más de un tipo.

Se pueden considerar tres tipos de polimorfismo (sobrecarga, inclusión paramétrica o polimorfismo dinámico y polimorfismo paramétrico) que se describen a continuación.

a) *Sobrecarga:* Según Booch [Booch, 1994], este tipo de polimorfismo fue inicialmente descrito por Strachey [Harland, 1985] quien hablaba de un polimorfismo "ad hoc", por el cual símbolos tales como "+", "-", "*", etc., se podrían definir con varios significados.

   Hoy en día, en los lenguajes de programación actuales este concepto recibe el nombre de *sobrecarga*, en que una función admite diferente número de argumentos y se pueden definir tantas versiones de dicha función como posibles valores pueda tener.

***Ejemplo 2.10: Funciones "árearectángulo" e "imprimir" sobrecargadas***

```
Ej1:  arearectangulo (x0,y0,x1,y1) = (x1-x0)*(y1-y0)
      arearectangulo (x,y) = x*y
Ej2:  void imprimir (int);
      void imprimir (char);
      void imprimir (int, float);
```

*El uso de la función imprimir sería el siguiente:*

```
void main()
{
   int i;
   float f;
   char c;
   ...
   imprimir(i);
   imprimir(c);
   imprimir(i,f);
}
```

b) *Polimorfismo dinámico o inclusión paramétrica:* Un objeto, función o método viene determinado dinámicamente dependiendo de la clase de objeto que recibe el mensaje.

Se aplica cuando existe una clase (por ejemplo, "Empleado") que es una subclase de otra ("Persona"). En este caso, se podrá utilizar un objeto de la clase derivada de la misma forma que un objeto de la clase padre.

Por ejemplo, en la figura 2-1, la clase "Empleado" define un subtipo de "Persona" ya que un empleado cumple con todo el comportamiento de una persona (lo que sucede es que tiene un comportamiento adicional debido a que es un empleado de una empresa). Y se podrán utilizar objetos de tipo "Empleado" en cualquier sitio que admita objetos del tipo principal ("Persona").

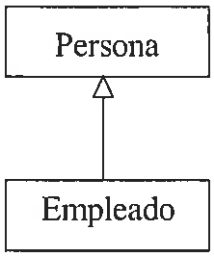



*Figura 2-1. Clase Empleado (subclase de Persona)*

***Ejemplo 2.11: Polimorfismo dinámico entre Persona y Empleado***

```
class Persona {
protected:
   char* nombre, docimilio, estado;
public:
   Persona(char* nombre, char* domicilio, char* estado) {.....}
      virtual char* Nombre() {return nombre; }
   ..........
};
class Empleado: public Persona {
    char* empresa;
public:
   Empleado(char* nombre, char* domicilio, char* estado,
      char* empresa){.....}
      char* Nombre() {return nombre; }
      ..........
};
```

Aplicando polimorfismo dinámico se podrá crear un Empleado y utilizarlo como si fuera una Persona.

```
Persona per ("Juan","Mayor,36",soltero); // Crea Persona per
Empleado emp("Luis","Arenal,4","casado", "Movistar");//Crea
Empleado emp
per.Nombre()// Obtiene "Juan"
emp.Nombre()// Obtiene "Luis"
Persona *pr=&emp; // puntero de tipo Personona que apunta al
Empleado emp
```

```
pr->Nombre() ; //Obtiene Luis
```

c) *Polimorfismo paramétrico o genericidad*: Strachy también define este tipo de polimorfismo, por el cual un elemento (una función, un operador, una clase, etc.) se define como una plantilla que implementa la misma ejecución pero admitiendo diferentes tipos de datos. En el caso de una función, sus argumentos se pueden aplicar a más de un tipo de dato

   En C++ se podría definir la función *imprimir* mediante polimorfismo paramétrico de la siguiente manera:

***Ejemplo 2.12: Definición de la función imprimir mediante polimorfismo paramétrico***

```
template <class T>
void imprimir (T x) {
   // Código de la función, que usa el dato "x" que
   // puede ser de cualquier tipo.
}
```

La utilización de esta función consistiría en llamarla con un parámetro apropiado a la función pero de cualquier tipo:

```
...
imprimir(20); // llamada con un entero
imprimir(3.4); // Número real
imprimir(true); // Valor booleano
...
```

El polimorfismo de este tipo permite compartir código, ya que basta con definir una única vez el cuerpo de la función para que esa función pueda aplicarse a cualquier tipo de datos (siempre y cuando las operaciones que se necesiten en el cuerpo de la función sean aplicables a datos de ese tipo).

El polimorfismo paramétrico se conoce también con el nombre de ***genericidad*** que es "la capacidad que tiene un lenguaje de programación de formar funciones o clases paramétricas (arrays, listas, árboles, etc.) que describen estructuras de datos genéricas".

En el ejemplo anterior en C++, la función "imprimir" sería una función con polimorfismo paramétrico y el argumento "x" es un parámetro genérico formal.

El ejemplo 2.13 describe, en el lenguaje funcional Hope, la función con polimorfismo paramétrico "arearectangulo". En este lenguaje *alpha* y *beta* son variables genéricas.

***Ejemplo 2.13: Definición de la función "arearectángulo" mediante polimorfismo paramétrico***

```
dec arearectangulo: alpha#alpha -> alpha
--- arearectangulo (x,y) <= x*y;
Ejecución: >> arearectangulo(12,5); Resultado: >>: 60
Ejecución: >> arearectangulo(23.2,12.7); Resultado: >>: 294.64
```

En el ejemplo 2.14 se codifica en Hope: a) la función no polimórfica *pertenecenum* que indica si un número pertenece o no una lista de números; b) la función *pertenece* con polimorfismo paramétrico que realiza la misma función pero con diferentes tipos de datos.

***Ejemplo 2.14: Definición de las funciones "pertenecenum" y "pertenece"***

*a) Función "pertenecenum" que indica si un número pertenece a una lista de números*

```
dec pertenecenum: num # list(num) -> truval;
--- pertenecenum(x,nil) <= false;
--- pertenecenum(x,a::l) <= if x=a then true else
                            pertenecenum(x,l);

Ejecución:>> pertenece(6,[2, 3, 4, 5]); Resultado>>: false
```

*b) Función "pertenece" que indica si un dato pertenece a una lista de elementos del mismo tipo*

```
dec pertenece: alpha # list(alpha) -> truval;
--- pertenece(x,nil) <= false;
--- pertenece(x,a::l) <= if x=a then true else pertenece(x,l);

Ejecución:>> pertenece(4,[2, 3, 4, 5]; Resultado: >>: trae

Ejecución:>> pertenece ('e', "hola"); Resultado: >>: false
```

Como podemos comprobar con los ejemplos anteriores, el polimorfismo paramétrico es una característica natural del paradigma funcional mientras que su inclusión en el paradigma procedimental resulta más forzada.

## 2.9 Evaluación Retardada o Diferida

Al evaluar una función existen dos tipos de evaluación:

a) *Evaluación ansiosa ("eager evaluation").-* En este tipo de evaluación se evalúan los argumentos actuales y se sustituye su valor por cada ocurrencia de los formales. La aplicación de una función a un argumento se realiza evaluando previamente el argumento antes de pasarlo a la función, lo que se denomina *llamada por valor* ("*call by value*"). Este es el tipo de evaluación convencional y es una evaluación de *1$^{er}$ orden.*

   La ventaja que tiene es que es fácil de implementar eficientemente, mientras que presenta el inconveniente de que se evalúen innecesariamente los argumentos si por alguna circunstancia no son requeridos en la ejecución de la función.

b) *Evaluación retardada o diferida ("lazy evaluation").-* En este tipo de evaluación los argumentos de las funciones no se evalúan hasta que no se necesitan. Es la *llamada por necesidad* ("*call by need*") en la que todos los argumentos de la función son pasados al cuerpo de la función sin evaluación previa, y sólo son evaluados cuando el cuerpo de la función los necesita. Es una evaluación de *2º orden.*
   Esta alternativa presenta la desventaja de que su implementación es más cara porque se debe pasar a la función una expresión sin evaluar en lugar de un valor simple como en el caso de la *llamada por valor.* Pero presenta otras potencialidades como la creación de estructuras infinitas que veremos más adelante.

El hecho de que el paradigma funcional utilice como elemento fundamental de la iteración la *recursividad* es lo que provoca la conveniencia (y en algunos casos la necesidad) del modelo de evaluación diferida.

Por otro lado, la evaluación diferida se ajusta mal al paradigma procedimental debido al efecto de lado. Así, en el ejemplo 2.15 se describe la función "*cuadrado*" que en el caso a) se comporta igual con ambos tipos de evaluación, pero en el caso b) los resultados son diferentes porque cuando se lee el fichero f se produce un efecto de lado al pasar de un registro al siguiente.

***Ejemplo 2.15: Evaluación de la función "cuadrado" con ambos tipos de evaluación***

***cuadrado (n) = n*n***

```
a)  cuadrado(p+q);  p=2, q=5
    Evaluación ansiosa: cuadrado (2+5) = 7*7 = 49
    Evaluación diferida: cuadrado(2+5) = (2+5) *(2+5) =7*(2+5) =7*7 = 49
b) cuadrado(getint(f));  f= 2, 5, 6, ….
    Evaluación ansiosa: cuadrado (2) = 2*2 = 4
    Evaluación diferida: cuadrado(getint(f)) = getint(f)* getint(f) = 2
     * getint(f) = 2* 5 = 10
```

La razón por la que la evaluación diferida no se ajusta bien al paradigma procedimental es que el programador tiene poco control sobre el orden de evaluación. Una función que construye un valor compuesto a partir de componentes simples no incide en el orden en que sus componentes son evaluados, ello viene determinado por el proceso de cálculo que *usa* el valor compuesto. Pero una falta de control en el orden de la evaluación puede producir graves consecuencias si como en el ejemplo anterior, existe efecto de lado, como ocurre en el paradigma procedimental.

Por otra parte, el procesamiento de estructuras de datos infinitos no sería posible sin aplicar la evaluación diferida.

El ejemplo 2.16 presenta en Hope dos funciones: "*lista*" que forma una lista infinita de números; y "*primerimpar*" que obtiene el primer número impar de una lista. Como se vé, *primerimpar(lista(8))* produce, con evaluación ansiosa, una iteración infinita, mientras que obtiene el valor 9 al aplicar evaluación diferida.

***Ejemplo 2.16: Obtener el primer número impar de una lista infinita***

```
dec lista: num  -> list (num) ;
--- lista (n) <= n::lista(n+1);

dec primerimpar: list (num) -> num;
--- primerimpar (nil) <= 0 ;
--- primerimpar (x::l) <= if x  mod  2 /=  0 then x else
                          primerimpar ( l ) ;

>> primerimpar(lista(8));

Evaluación ansiosa: pi(8::lista(9)) = pi(8::9::lista(10) = …..

Evaluación diferida: pi(lista (8) ) = pi (8:: lista (9) )=
pi(lista (9)) = pi (9:: lista (10) ) = 9
```

La evaluación diferida suele ser necesaria cuando se utilizan funciones constructoras como en el ejemplo anterior.

No todos los lenguajes de programación funcional aplican la evaluación diferida. El lenguaje Hope tiene dos implementaciones: una estándar en la que las funciones constructoras son *llamadas por necesidad* y el resto de las funciones *por valor*, y otra diferida en que todas las funciones son *llamadas por necesidad*. Mientras que el lenguaje Lisp no tiene evaluación diferida, por lo que no se podría codificar el ejemplo anterior en este lenguaje. El lenguaje ML usa evaluación ansiosa, mientras que el dialecto "Lazy ML" soporta evaluación diferida. El lenguaje Haskell también realiza evaluación diferida.

Un beneficio de la evaluación diferida es el hecho de soportar una nueva clase de modularidad: la separación del *control* del *cálculo*. La idea es romper un cálculo iterativo o recursivo en una parte de cálculo puro, que computa todos los valores necesarios y construye un valor compuesto (ej. una lista infinita o diferida), y otra parte que recorre todo o parte de ese valor compuesto determinando así el flujo de control. A menudo, se puede expresar el cálculo de una forma más simple una vez que la información de control ha sido eliminada. A menudo, la misma parte de cálculo se puede reusar con diferentes partes de control, o una parte de control se puede reusar con diferentes partes de cálculo (Watt, 1990).

## 2.10 Mónadas

El lenguaje Haskell, modelo del paradigma funcional moderno, incorpora el concepto de mónada. Las mónadas son elementos encapsulados que proporcionan características procedimentales sin perder los beneficios del paradigma funcional: transparencia referencial, funciones de orden superior y evaluación diferida. Y proporcionan un desarrollo estructurado y secuencial.

Su estructura simple consta de un *constructor de tipo unario* (m) y dos funciones polimórficas:

- Una función (ej. return, result, etc.) toma un valor de un tipo *a* y crea un ítem mónadico *ma*. Es decir, una función que inyecta el tipo de data *a* en la mónada *ma* :

```
Ej.- return:: a -> ma
```

- Otra función (ej. (>>=), then, bind, etc.) toma el item *ma* y una función que transforma un valor del tipo *a* a otro tipo monádico *mb*, y retorna un item monádico del tipo *mb*. Es decir, enlaza dos funciones monádicas, pasando el resultado de la primera a la segunda y devolviendo el resultado de la segunda

```
Ej.- (>>=):: ma -> (a-> mb) -> mb
```

Resumiendo tenemos, un tipo de dato *a*, que puede contener cualquier tipo de dato, una función (>>=) que combina dos funciones que procesan el tipo monádico, devolviendo como resultado otra instancia del tipo monádico y una función (*return*) que, dado un tipo, nos devuelve el tipo encerrado en la mónada.

Además, el tipo y funciones definidos en la mónada deben cumplir las 3 leyes monádicas, que vienen a decir, de manera informal, que las funciones (>>=) y (*return*) deben comportarse bien con el elemento identidad y ser asociativas.

El desarrollo monádico permite una estrategia de evaluación secuencial. Y sustituyen funciones del tipo *"values to values"* por funciones "*values to computations*".

*Ejemplo 2.17: Mónada que obtiene el cuadrado de cada elemento de una lista de enteros incrementados en 10*

```
CuadradoListaMasDiez:: [Int] -> [Int]
CuadradoListaMasDiez xl
      = xl              >>=\ x ->
      return (square x)       >>=\ y ->
      return (y+10)
Entrada: CuadradoListaMasDiez [2, 3, 4, 5]      Salida:
=>[14,19,26,35]
```

Las mónadas se usan en Haskell: para modelar la entrada/salida sin perder pureza funcional, para modelar computaciones con estado mutable como las de un parser, para las excepciones, etc..

Haskell encierra las acciones impuras de entrada/salida en una mónada especial, *la mónada IO*. De esta manera, las funciones que usan esta mónada siguen siendo puras, recibiendo y devolviendo valores de la mónada IO que describen una "acción de entrada salida", pero sin que su pureza se vea afectada por los efectos laterales. Además la mónada permite que la programación funcional sea interactiva.

*Ejemplo 2.18: Mónada 10 que lee dos caracteres y los imprime*

```
exIO:: IO()
exIO =      getChar     >>=\ c1 ->
getChar           >>=\ c2 ->
putChar c2        >>=\ _
putChar c1
```

En la definición de mónada vimos que se definía también la función (>>=) para enlazar dos funciones. Esto es muy útil porque en Haskell se usa la "evaluación diferida", por lo que no se puede decidir en qué orden se van a ejecutar las llamadas a las funciones. Por ejemplo en (20 * (12 + 8)), no se sabe si se va a usar la propiedad distributiva o no, así que para secuenciar llamadas a funciones hay que utilizar funciones como (>>=) que permiten enlazar la ejecución secuencial de una serie de funciones.

## 2.11 Tipos de Datos en el Paradigma Funcional

En general, el paradigma funcional utiliza muy pocos tipos de datos *primarios o básicos*. Los más usuales son:

a) *Datos atómicos*, que son números enteros, reales, booleanos y caracteres. Ejemplo: 28, 43.76, T (cierto) ó NIL (falso), Programación, 'h'.

b) *Identificadores*, que son los nombres de las variables y de las funciones.

   Ejemplo: x, suma(x).

c) *Listas*. Conjuntos de elementos que corresponden a un tema común. Las cadenas de caracteres se consideran en general como listas de caracteres.

Ejemplo: La lista de alumnos de una clase (Luis_Pérez, Andrés_Martínez, Rosa_Gómez....), la cadena "hola" (equivalente a la lista ('h', 'o', 'l', 'a').

Al ser una *lista* un conjunto de elementos en el que va uno detrás de otro, su acceso siempre será al primer elemento de la lista, denominado la *cabeza*; siendo el resto de los elementos la cola de la lista.

Como el paradigma funcional maneja mucho las listas, sus lenguajes tienen funciones ya definidas que obtienen la *cabeza* o la *cola* de una lista. Esto se puede comprobar en lenguajes funcionales tales como Lisp y Hope donde se describe el problema de: "*Programar* la función *Sumar* que suma todos los números existentes en una lista L que recibe como argumento"

***Ejemplo 2.19: Función sumar que suma todos los números existentes en una lista L***

***Codificación en Lisp:***

```
(DEFUN SUMAR(L)
       (COND ((NULL (CDR L)) (CAR L))
             (T (PLUS (CAR L) (SUMAR (CDR L))))))
```

**NOTA**:

- DEFUN: Define una función.
- COND: Es la función condición: if ... then ... else.
- CAR: Obtiene la cabeza de la lista.
- CDR: Obtiene la cola de la lista.
- PLUS: Es la función +

Si se define L= (4,8,5,10), al ejecutar Sumar (L) mostrará por pantalla el valor 27.

La conversación entre el usuario y el programador sería:

```
? L = (4,8,5,10)
? Sumar (L)
27
```

***Codificación en Hope:***

```
dec Sumar: list(num)-> num;
--- Sumar(nil) <= 0;
--- Sumar(x::l) <= x + Sumar(l);
```

- de números y proporciona como resultado un número.
- *x::l* : define una lista cuya cabeza es x y la cola l.

## 2.12 Programas Funcionales

Un programa funcional es pues, "*un conjunto compuesto de funciones que juntas definen un objetivo específico de aplicación*".

Un programa funcional viene definido por la ecuación:

*Funciones + Estructuras de Datos = Programa*

Siendo sus elementos fundamentales:

a) **Las funciones**, elementos básicos del paradigma. Poseen las propiedades de ser de orden superior y con transparencia referencial.

b) **La recursividad**, que es la iteración con asignación no destructiva. El carácter recursivo de esta programación exige disponer de este mecanismo, especialmente utilizado en la construcción de funciones.

c) **Las listas**, que como se ha indicado anteriormente, son las estructuras básicas de almacenamiento de la información. Aunque la programación funcional utiliza también otros tipos de estructuras, se fundamenta básicamente en el manejo de información simbólica con estructura en lista. En este tipo de programación, la expresión algebraica carece de la relevancia que tiene en la programación imperativa.

Peter Bishop [Bishop, 1986] señala que los lenguajes funcionales son aplicativos porque aplican funciones, y pertenecen al conjunto de los denominados lenguajes de programación funcional surgidos del cálculo de la función lambda de Church. Un programa en un lenguaje funcional es considerado como un conjunto de *descripciones* de objetos de datos, más que como un conjunto de *almacenamientos* que proporciona los resultados del cómputo de estos datos.

En concreto, los lenguajes funcionales, tipo Lisp, Hope, Haskel, etc. constan de:

- *Funciones primitivas* predefinidas en el lenguaje: de control, de estructuras, aritméticas...
- *Formas funcionales* para crear y combinar funciones.
- *Mecanismos para aplicar los argumentos* y producir valores de funciones; uno de estos mecanismos es la currificación [Curry, 1958] que consiste en tratar una función de n argumentos como una concatenación de n funciones con un solo argumento. Ej.: *f x y z => (((f x) y) z)*.
- *Objetos de datos* del tipo átomos, listas, etc.
- Los primeros lenguajes eran *interpretados* (no generan un ejecutable), el intérprete va interpretando (traduciendo) el código, línea a línea, excepto aquellas funciones ya compiladas en lenguaje máquina que directamente son llamadas sin necesidad de traducción. Actualmente suelen ser lenguajes compilados porque la compilación ahorra tiempo de ejecución al optimizar su ejecución.
- Son *aplicativos*, ya que se fundamentan en la aplicación progresiva de estas funciones.

## LISP

Como lenguaje pionero de la programación funcional vamos a describir brevemente este lenguaje. Lenguajes actuales de tipo funcional como el Haskell y el Hope se describirán en la siguiente unidad.

El lenguaje LISP ("List Processing") diseñado por John McCarthy, en 1959, en el Instituto Tecnológico de Massachusetts es un lenguaje híbrido (funcional y procedimental) que incorpora en su expresión más simple los siguientes elementos:

a) *Funciones primitivas predefinidas*.- Fundamentalmente son las siguientes:

- *Tratamiento de* listas:
- CAR (devuelve la cabeza de la lista); CDR (devuelve la cola de la lista); CONS (construye una lista a partir de dos parámetros: cabeza y cola); LIST (construye una lista a partir de varias sublistas).

  Ejemplos:

```
(CAR ' ((A B) C))→ (A B)
(CDR ' ((A B) C))→ ( C )
(CADDR ' ((A B C) D E))→ E
(CONS ' A  ' (B C))→ (A • (B C)) → (A B C)
(LIST ' A  ' B  ' C)→ (A B C)
```

**NOTA**: CADDR es un CDR, CDR y CAR en la misma instrucción

- *Asignación, relaciones booleanas y predicados*
  - SET y SETQ.- Evalúa sus argumentos, (en el caso de SETQ sólo evalúa el 2º argumento) y asigna el valor del segundo argumento al primero. Son dos funciones no declarativas, porque realizan una asignación destructiva.
  - EQ.- Comprueba si dos argumentos son iguales
  - GREATERP y LESSP.- Comparan si el primer argumento es mayor/menor que el segundo
  - NULL.- Comprueba si la lista es nula
  - ATOM, LISTP y NUMBERP.- Averiguan el tipo (átomo, lista o número) de un objeto
  - OR, AND y NOT.- Predicados de suma, multiplicación y negación booleana.

  Ejemplos:

```
(SETQ  X  'programacion) → Asigna "programación" a la
variable X
(EQ X  'programacion)→ T
(NULL  X)→ F
(ATOM  X)→ T
(NOT (OR (EQ 3  4) (EQ X  'programacion)))→ NIL
```

- *Aritméticas*

- PLUS, DIFFERENCE, TIMES, QUOTIENT, REMAINDER (suma, resta, multiplicación, cociente, resto)

Ejemplos:

```
(PLUS 3  4)→ 7
(DIFFERENCE 10  20)→ -10
(TIMES 3 - 15)→ -45
(QUOTIENT 17  3)→ 5
(REMAINDER 17  3)→ 2
```

- *Condicionales e iterativas*

  - COND.- Función condicional con varios predicados. Su formato es:
    (COND (predicado1 exp1) ... (predicadon expn) )
    - Si se verifica un predicado se ejecuta la expresión y finaliza la función condicional
  - LOOP.- Función iterativa, que se incorporó posteriormente. Sus formatos son:
    (LOOP secuencia-operaciones)
    (LOOP (UNTIL predicado exp) secuencia-operaciones) )
    - Hasta que el *predicado* no sea igual a la *exp* se ejecuta *la secuencia-operaciones*. La pregunta del UNTIL se realiza al finalizar cada iteración
    (LOOP (WHILE predicado exp) (secuencia-operaciones) )
    - Mientras el *predicado* verifique la *exp* se ejecuta la *secuencia-operaciones*. La pregunta del WHILE se realiza al principio de cada iteración.

Ejemplo:

```
(COND ((EQ A B) 20) (T 30)
equivale a: si A = B entonces 20 si no 30.
```

Ejemplo:

```
(SETQ Y 0)
(LOOP (UNTIL (GREATERP Y 100)
        (SETQ Y (PLUS  Y  5))
        (PRINT Y))
```

En lenguaje C sería:

```
y = 0;
do{
    y = y + 5;
    cout << y;
}while (y <= 100);
Imprime 5  10  15 ... 100  105
```

b) *Formas funcionales para crear y combinar funciones*

- DEFUN.- Función que permite crear otras funciones. Su formato es:

  (DEFUN nombre-función (lista-argumentos) cuerpo-función)

  Ejemplo: función Doble(x)=x*2

```
(DEFUN DOBLE (X) (TIMES X 2) )
> (DOBLE 14)→ 28
```

c) *Objetos de datos*

LISP, como lenguaje orientado a listas, dispone de esta estructura como fundamental.

En general, un programa en LISP utiliza:

- *Átomos*: números y combinación de caracteres y dígitos.

  Ej.: 28, programación, -27, valor_total

- *Identificadores*: nombres de variables o de funciones

  Ej.: X, DOBLE, Y25

- *Listas*: son átomos y/o sublistas encerrados entre paréntesis

  Ej.: (20 4 35) (X Y (A B ( C ) D E))

A continuación se presentan dos ejemplos en LISP. El primero permite comprobar si un número es primo o no; y el segundo ordena ascendentemente una lista de datos.

***Ejemplo 2.20: Primo***

```
;; Determinar si un número es primo o no
(DEFUN PRIMO (N)
   (COND ((EQ N 2) T)  ((EQ N 3)  T)
         (T (PRIMO1 N (QUOTIENT N 2)))))
(DEFUN PRIMO1 (N  I)
   (COND ((EQ (REMAINDER  N  I)  0)  NIL)
         ((EQ I  2) T)
         (T (PRIMO1  N (DIFFERENCE  I 1)))))
```

Ejecución:

```
(PRIMO 3)→ T
(PRIMO 8)→ NIL
```

***Ejemplo 2.21: Ordenar***

```
;; Ordenar ascendentemente los elementos de una lista
;; INSERCIÓN: Dividir la lista en el 1° elemento y el resto
;; A continuación, insertar el primero en lo ordenado del resto
   ( DEFUN   ORDENAR (LISTA)
      (COND (( NULL LISTA ) NIL)
         (T ( INSERTAR ( CAR LISTA )
                  ( ORDENAR ( CDR LISTA )))))
   ( DEFUN INSERTAR ( X LISTA )
      ( COND (( NULL LISTA ) X )
      ( T ( COND (( <= X ( CAR LISTA ))(CONS X LISTA ))
            ( T ( CONS ( CAR LISTA )
                  ( INSERTAR X ( CDR LISTA ))))))
```

Ejecución:

```
(ORDENAR ( 4  3  8  7  20) ) → ( 3  4  7  8  20 )
```

Aunque el lenguaje LISP fue el primer lenguaje que implementa la programación funcional, no es un lenguaje paradigmático de este modelo de programación ya que incluye conceptos no funcionales tales como: efecto de lado con modificación destructiva de su contenido, tipos de datos con almacenamiento fijo (arrays y otros objetos manipulados básicamente con efecto de lado), constructores del flujo de control (por ejemplo, iteraciones y manejo de excepciones), etc. que lo hacen un lenguaje híbrido (declarativo y procedimental) en el que la secuencia de control juega un papel importante.

## 2.13 Ventajas e Inconvenientes del Paradigma Funcional en Relación con el Imperativo

Las ventajas del paradigma funcional son:

- *Simplicidad*: el modelo de cómputo funcional y sus lenguajes son más simples que los imperativos. Por ejemplo, el lenguaje Haskell tiene 23 identificadores y el manual 80 páginas, mientras que Ada tiene 62 identificadores y el manual 200 páginas.

- *Posibilidad de razonamiento formal:* la transparencia referencial permite razonamiento ecuacional (sustituir iguales por iguales) en el paradigma funcional. En el paradigma imperativo ha de utilizarse la lógica de predicados.

- *Concisión:* la relación del número de líneas de código entre el paradigma imperativo y el paradigma funcional es de 10 a 1. Lo que indica que un código más reducido permite obtener un programa más conciso y por lo tanto más fiable.

- *Abstracción:* El paradigma funcional se ajusta mejor al modelo del problema y peor al modelo del computador que el paradigma imperativo, lo que permite obtener un código más reutilizable.

- *Paralelismo:* la ausencia de efecto de lado del paradigma funcional facilita el cálculo paralelo, por ello el este paradigma es ideal para la programación paralela y concurrente.

Por otro lado, el paradigma funcional presenta los siguientes inconvenientes:

- *Dificultad del Modelo Funcional:* el paradigma funcional es poco intuitivo para el programador cuya línea de pensamiento está más en cómo opera el computador que es la forma de ejecución del paradigma imperativo.

- *Ineficiencia:* el paradigma funcional es de 3 a 5 veces más lento en su ejecución que el paradigma imperativo.

- Actualmente el paradigma funcional no es apropiado para la *programación en gran escala.* Sí que es bueno para investigar. Aunque los lenguajes Haskell, Standar ML y Miranda empiezan a ser bastante eficientes, especialmente Haskell.

## 2.14 Ejercicios de Autocomprobación

*1.* ¿De qué dos formas se suele especificar una función? *Declarativamente (matemáticamente) o procedimentalmente.*

2. ¿Cómo se denomina en el paradigma funcional la característica de que una función para unos mismos valores de entrada sólo pueda producir un único valor de salida? *Transparencia referencial.*

3. ¿Qué características definen a una función de orden superior? *Que sus argumentos y su valor de retorno pueden ser otra función.*

4. ¿Sería válido utilizar un puntero en este paradigma? *No, porque un puntero incorpora el concepto de estado de memoria, lo que no es válido en el paradigma.*

5. ¿Qué se entiende por una función con polimorfismo paramétrico? *Que implementa la misma ejecución pero admitiendo para sus argumentos diferentes tipos de datos.*

6. ¿Qué tipos de llamadas se realizan en las evaluaciones ansiosa y diferida? *En la evaluación ansiosa se realiza una "llamada por valor"(call by value), mientras que en la diferida se realiza "llamada por necesidad" (call by need).*

7. ¿Qué es una mónada? *Es un elemento encapsulado que proporciona características procedimentales sin perder los beneficios del paradigma funcional.*

8. ¿Cuáles son los elementos fundamentales de un programa funcional? *Las funciones, la recursividad y las listas.*

9. Las mónadas son una característica básica de la programación funcional? *No. Las mónadas son elementos encapsulados que proporcionan características procedimentales sin perder los beneficios del paradigma funcional.*

10. ¿Enumera las ventajas del paradigma funcional en relación con el imperativo? *Simplicidad, posibilidad de razonamiento formal, concisión, abstracción y paralelismo.*

## 2.15 Conclusiones

Las conclusiones más importantes que se deducen del tema son las siguientes:

- El modelo de cómputo del paradigma funcional (PF) está basado en la lógica ecuacional: "un programa funcional consiste en un conjunto de definición de funciones cuyos patrones son disjuntos".
- El PF incorpora el concepto de función como objeto de primera clase lo que significa que las funciones pueden ser tratadas como datos.
- Para que puedan ser tratadas como datos es preciso que tengan transparencia referencial, es decir, sin efecto de lado.
- La transparencia referencial es indispensable para la composición de funciones. Reemplaza "valores por valores", es decir, sustituye iguales por iguales y permite verificar la corrección de programas.
- El PF utiliza funciones de orden superior: aceptan como parámetros otras funciones, pueden retornar como resultado otra función y se pueden evaluar como un objeto.
- En el PF no existe el concepto de estado (celda de memoria) por lo que: utiliza valores locales para los resultado de cálculos intermedios, no se puede utilizar la asignación destructiva (a← b) y no se pueden realizar procesos iterativos (do, while, for,..)

- El PF admite funciones con polimorfismo de sobrecarga, mediante la utilización de patrones disjuntos para la definición de las funciones, y con polimorfismo paramétrico, en que sus argumentos se pueden aplicar a más de un tipo de dato.
- El PF admite la evaluación diferida o perezosa donde los argumentos de las funciones no se evalúan hasta que no se necesitan. La evaluación diferida permite crear y tratar listas infinitas. La evaluación diferida se ajusta mal a los lenguajes imperativos debido al efecto de lado
- El lenguaje Haskell implementa las mónadas que son elementos encapsulados que proporcionan a la programación funcional un desarrollo estructurado y secuencial.

## 2.16 Bibliografía


- **ALONSO, F.; MARTÍNEZ, L.; SEGOVIA, F.J.:** *Introducción a la Ingeniería del Software.- Modelos de Desarrollo de Programas*. Delta Publicaciones, 2005.
- **AMBLER, A.L.; BURNETT, M.; ZIMMERMAN, B.:** *"Operational Versus Definitional: A Perspective on Programming Paradigms"*. Sept./1992, IEEE Computer.
- **BAILEY R.: "A Hope Tutorial"** http://www.soi.city.ac.uk/~ross/Hope/hope_tut/hope_tut.html.
- **BISHOP, P.:** *"Fifth generation computers: Concepts, Implementation and Uses"*. Ellis Horwood, 1986.
- **BOOCH, G.:** *"Object-Oriented Analysis and Design with Applications"*. Benjamin Cummings, 1994.
- **CURRY, H.B.; FEYS, R.:** *"Combinatory Logic"*. Vol 1. North Holland. 1958.
- **FIELD, A.J; HARRISON, P.G.:** *"Functional Programming"*. Addison-Wesley Publishing, 1993.
- **GRAHAM P.:** *"ANSI Common Lisp"*. Prentice Hall, 1995.
- **HARLAND, D.** et al.: *"An Alternative View of Polymorphism"*. SIGPLAN Notices, vol. 20. October, 1985.
- **PEYTON S. J.:** *"Haskell 98 Language and Libraries: The Revised Report"*. Microsoft Research, Cambridge. 2002.
- **RUIZ, BALS C.:** *"Razonando Con Haskell: Un Curso Sobre Programación Funcional"*. S.A. Thomson Paraninfo, 2004.
- **WATT, D. A.:** *"Programming Language. Concepts and Paradigms"*. Ed. Prentice-Hall, *1990.*